Guia do Sistema — Assistência Técnica de Celulares

Como instalar, criar o banco de dados, rodar e testar o sistema, e o que faz cada arquivo do projeto.

Sumário

1. O que você precisa ter instalado
2. Extrair o projeto
3. Instalar as dependências do Python
4. Criar o banco de dados
5. Iniciar o sistema
6. Testar as funcionalidades
7. Gerar o DER no MySQL Workbench
8. Problemas comuns
9. O que é cada arquivo
10. Entrega

1. O que você precisa ter instalado

- **Python 3.10 ou mais novo**. Para conferir, digite `python --version` no PowerShell.
- **MySQL Server 8** funcionando, e a **senha do usuário root**.
- **MySQL Workbench**, para criar o banco e fazer o DER.

2. Extrair o projeto

1. Clique com o botão direito no arquivo `.zip` e escolha **Extrair tudo...** → **Extrair**.
2. Dentro da pasta extraída você vai ver: a pasta `sistema`, o arquivo `assistencia_db.sql`, o `der.png` / `der.pdf`, a documentação e este guia.

3. Instalar as dependências do Python

1. Abra a pasta `sistema`. Clique na **barra de endereço** do Explorador de Arquivos, digite `powershell` e aperte **Enter**. O PowerShell abre já dentro dessa pasta.
2. Rode:

```
pip install -r requirements.txt
```

Isso instala o **Flask** (servidor web), o **bcrypt** (criptografia de senha e CPF) e o **mysql-connector-python** (conexão com o MySQL).

> Você só precisa fazer isso **uma vez** em cada computador.

4. Criar o banco de dados

O arquivo `assistencia_db.sql` cria o banco `assistencia_db` com as 5 tabelas e 3 registros de teste em cada uma.

Opção A — pelo MySQL Workbench (mais fácil)

1. Abra o Workbench e entre na conexão **Local instance MySQL80** com a sua senha.
2. Menu **File** → **Open SQL Script...** e escolha o `assistencia_db.sql`.
3. Clique no **raio** ⚡ (Execute) para rodar o script inteiro.
4. No painel **Schemas**, à esquerda, clique em atualizar (🔄). O banco `assistencia_db` vai aparecer.

Opção B — pelo PowerShell

Na pasta onde está o `assistencia_db.sql`:

```
cmd /c "mysql -u root -p < assistencia_db.sql"
```

Ele pede a senha do root. Se der “mysql não é reconhecido”, use o caminho completo:

```
cmd /c "\"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe\" -u root -p < assistencia_db.sql"
```

> **Atenção:** o script começa com `DROP DATABASE IF EXISTS assistencia_db`. Se você rodar de novo, o banco é **apagado e criado do zero**, e os dados que você cadastrou se perdem.

5. Iniciar o sistema

1. Abra o PowerShell **dentro da pasta** `sistema`. É a pasta que tem o `app.py`.
2. Informe a senha do seu MySQL e inicie o sistema, tudo numa linha:

```
$env:DB_PASSWORD="SUA_SENHA_DO_MYSQL"; python app.py
```

3. Quando aparecer `Running on http://127.0.0.1:5000`, abra no navegador:

```
http://localhost:5000
```

4. Para parar o sistema, aperte **Ctrl + C** no PowerShell.

Para testar só a conexão com o banco, sem abrir o site:

```
$env:DB_PASSWORD="SUA_SENHA_DO_MYSQL"; python testar_conexao.py
```

O resultado esperado é `Conexão OK: {'banco': 'assistencia_db', 'versao': '8.x'}`.

Usuários de teste

Usuário	Senha	Nome exibido
`admin`	`admin123`	Administrador
`carlos`	`carlos123`	Carlos Técnico
`ana`	`ana12345`	Ana Atendente

6. Testar as funcionalidades

Tela	O que fazer	Resultado esperado
Login	Senha errada / campos vazios	"Usuário ou senha inválidos." / "Informe usuário e senha."
Login	`admin` / `admin123`	Abre a tela principal com "Bem-vindo(a), Administrador!"
Topo	Olhar a barra de cima	Mostra "Usuário: Administrador" e o botão **Sair**
Clientes	Buscar "maria" ou o CPF `111.444.777-35`	Lista só a Maria Oliveira
Clientes	Cadastrar com CPF `123.456.789-09`	Aparece na lista; o CPF é salvo criptografado
Clientes	Editar → mudar telefone → Salvar	Telefone atualizado na lista

Aparelhos	Escolher o cliente, IMEI `352099001761481`, marca, modelo, defeito	Aparelho salvo, ligado ao cliente
Aparelhos	IMEI `12345` ou um repetido	"IMEI inválido..." / "IMEI já cadastrado."
Agendamentos	Abrir a tela	Lista ordenada por data, com dados do cliente e do aparelho
Agendamentos	Reagendar → nova data → Salvar	Data atualizada e lista reordenada
Sair	Clicar em Sair	Volta ao login; abrir `/clientes` manda de volta ao login

Para conferir no Workbench que o CPF está criptografado:

```
SELECT nome, cpf_hash FROM assistencia_db.clientes;
```

O `cpf_hash` deve começar com `$2b$10$`.

7. Gerar o DER no MySQL Workbench

1. Com o banco já criado, vá em **Database** → **Reverse Engineer...**
2. Escolha a conexão local, clique em **Next** e digite a senha.
3. Marque o schema `assistencia_db` e vá clicando em **Next** até **Execute** → **Finish**.
4. O diagrama aparece com as tabelas ligadas pelas chaves estrangeiras. Arraste as tabelas para organizar.
5. Para exportar, use **File** → **Export** → **Export as PNG...** e salve como `der.png`.

Relacionamentos que devem aparecer:

- `clientes` 1 — N `aparelhos` (um cliente pode ter vários aparelhos)
- `aparelhos` 1 — N `agendamentos` (um aparelho pode ter vários reparos agendados)
- `servicos` 1 — N `agendamentos` (um serviço pode estar em vários agendamentos)
- `usuarios` fica sozinha: guarda só quem faz login no sistema

8. Problemas comuns

Mensagem	Causa	Solução
`can't open file ... app.py`	O PowerShell não está na pasta `sistema`	Rode `cd sistema` e tente de novo

`No module named 'flask'`	Dependências não instaladas	`pip install -r requirements.txt`
`Access denied for user 'root'`	Senha do MySQL errada ou não informada	Confira o `$env:DB_PASSWORD="..."`
`Unknown database 'assistencia_db'`	O banco não foi criado	Faça o passo 4
`Can't connect to MySQL server`	O serviço do MySQL está desligado	Abra *Serviços* do Windows e inicie o **MySQL80**
"Erro ao acessar o banco de dados." na tela	Qualquer um dos problemas acima	Rode `python testar_conexao.py` para ver o erro exato
Porta 5000 em uso	O sistema já está aberto em outro PowerShell	Feche o outro com **Ctrl + C**

9. O que é cada arquivo

Arquivos da entrega (raiz do zip)

Arquivo	Para que serve
`assistencia_db.sql`	Script que cria o banco `assistencia_db`, as 5 tabelas (usuarios, clientes, aparelhos, servicos, agendamentos) com chaves primárias e estrangeiras, e insere 3 registros de teste em cada uma.
`der.png` / `der.pdf`	Diagrama Entidade-Relacionamento do banco.
`documentacao.docx` / `.pdf`	Documentação de Software (Anexo 1): requisitos funcionais e requisitos de infraestrutura. Coloque o seu nome e o número da prova.
`GUIA.html` / `GUIA.pdf`	Este guia.
`sistema/`	Código-fonte do sistema.

Back-end em Python (pasta `sistema`)

Arquivo	Para que serve
`app.py`	Ponto de entrada. Cria o app Flask, configura a sessão (cookie seguro, 30 min) e sobe o servidor na porta 5000. É ele que você roda com `python app.py`.

`db.py`	Conexão com o MySQL. Lê `DB_HOST`, `DB_USER`, `DB_PASSWORD` e `DB_NAME` das variáveis de ambiente (padrão: localhost, root, sem senha, assistencia_db). Tem as funções `query` (SELECT) e `execute` (INSERT/UPDATE/DELETE), que usam parâmetros `%s` contra SQL Injection, e a classe `ErroNegocio` para as mensagens de erro.
`autenticacao.py`	Login e sessão: gera e confere hashes **bcrypt** (usados na senha e no CPF), valida usuário e senha, inicia a sessão com o nome do usuário e controla a **expiração de 30 minutos** (`EXPIRACAO_SEGUNDOS`).
`rotas.py`	Todas as URLs do sistema. Bloqueia quem não está logado, entrega as páginas HTML e tem a API JSON: `/api/login`, `/api/logout`, `/api/me` (nome do usuário logado), `/api/teste-conexao`, `/api/servicos` e o CRUD genérico `/api/clientes`, `/api/aparelhos` e `/api/agendamentos`.
`clientes.py`	Regras de clientes: valida nome, e-mail e CPF (11 dígitos, sem repetir), guarda o CPF **só como hash** (LGPD), faz a **busca** por nome, e-mail, telefone ou CPF, e cadastra, edita e exclui (não deixa excluir cliente com aparelho).
`aparelhos.py`	Regras de aparelhos: valida IMEI (15 dígitos, único), marca, modelo e defeito, e exige um **cliente já cadastrado**. A listagem faz `JOIN` com clientes para mostrar o nome do dono. Não deixa excluir aparelho com agendamento.
`agendamento.py`	Regras de agendamentos: lista **ordenada por data** com todos os dados do cliente, do aparelho e do serviço (JOIN de 4 tabelas), cria agendamento, **reagenda** (muda a data e o status) e exclui. Também lista os serviços.
`testar_conexao.py`	Script rápido para testar se o Python consegue conectar no MySQL.
`requirements.txt`	Lista das bibliotecas Python: Flask, bcrypt e mysql-connector-python.
`ROTEIRO.md`	Roteiro original do professor, com o passo a passo e os testes esperados.

Front-end (pastas `templates` e `static`)

Arquivo	Para que serve
`templates/login.html`	Tela de login. Envia usuário e senha para `/api/login` e mostra o erro se falhar.
`templates/principal.html`	Tela principal: “Bem-vindo(a), nome!” e os atalhos para Clientes, Aparelhos e Agendamentos.
`templates/clientes.html`	Campo de **busca**, formulário de cadastro e edição, e a tabela de clientes com Editar e Excluir.
`templates/aparelhos.html`	Formulário com a lista de **clientes** para associar o aparelho, mais a tabela de aparelhos.

`templates/agendamentos.html`	Formulário de novo agendamento, quadro de **reagendar** e a lista ordenada por data.
`static/app.js`	Funções JavaScript usadas por todas as telas: `api()` chama o servidor (e volta ao login se a sessão expirou), `montarMenu()` monta a barra do topo com o **nome do usuário** e o botão **Sair**, `sair()` faz o logout, e há também as funções de tabela, formulário e mensagens.
`static/style.css`	Visual do sistema: cores, barra do topo, cartões, formulários, botões e tabelas.

O que foi completado no desafio

Item	Onde
Conexão com o banco	`db.py` : host `localhost` e banco `assistencia_db`
Login	`rotas.py` : lê os campos `login` e `senha`
Nome do usuário	`static/app.js` → `montarMenu()` mostra "Usuário: nome"
Buscar clientes	`templates/clientes.html` : formulário de busca
Logout / botão Sair	`static/app.js` → função `sair()` e botão no topo
Associar cliente ao aparelho	`templates/aparelhos.html` (select `cliente_id`) e `aparelhos.py` (`JOIN clientes`)

10. Entrega

Crie um `.zip` na **Área de Trabalho** com o **seu nome completo**, contendo:

- `documentacao.pdf` , com o seu nome e o número da prova
- `der.png` ou `der.pdf`
- `assistencia_db.sql`
- a pasta `sistema`